

# Security Communication – Report

Reported By : Muhammed Shafi B

## 1.Title

Secure communication Practical Investigation

## 2.Objective

To practically understand HTTPS traffic, TLS certificates, hashing, and encryption using Wireshark, OpenSSL, Python, Power shell, Cyberchef

## 3.Tools Used

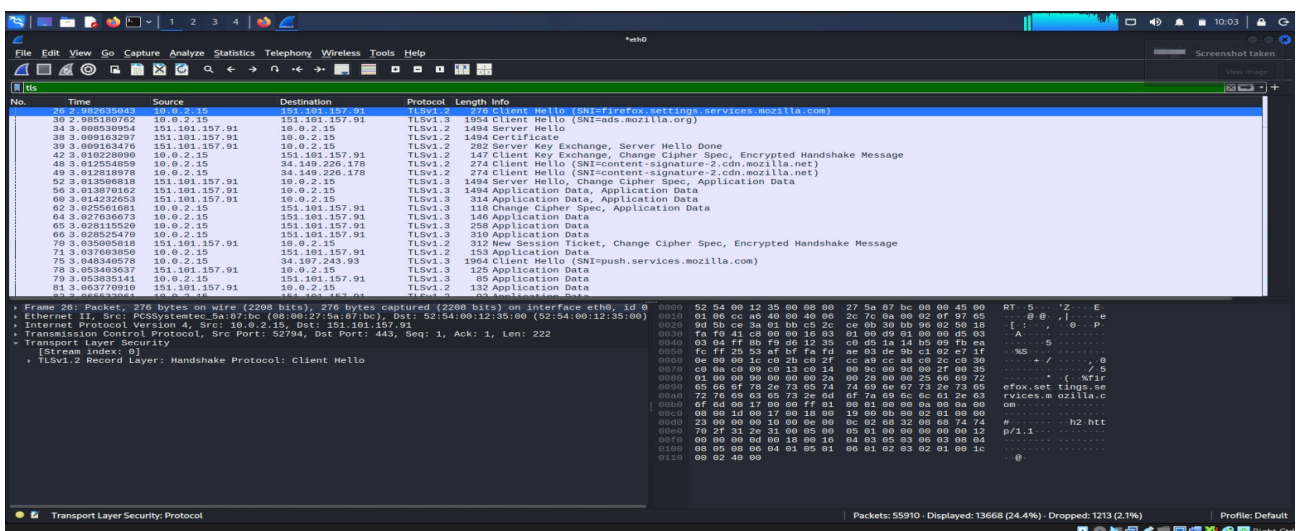
- Wireshark
- Powershell
- Python
- Cyberchef

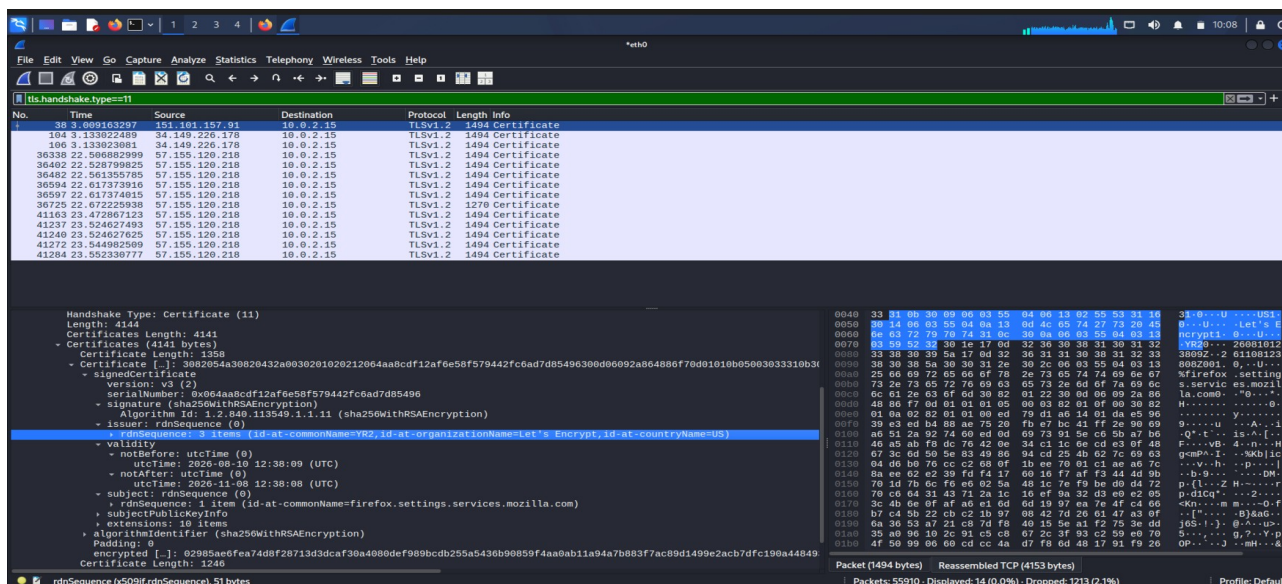
## 4. Analyse HTTPS Traffic

I analysed HTTPS traffic using Wireshark by capturing packets while opening an HTTPS website. The finding was that TLS encrypted the HTTP data, TLS version, Server name indicator, certificate information, encrypted application and TLS handshake packets were still visible. This shows how HTTPS protects data during communication.

Website :- flipkart.com

|                     |   |                                                                        |
|---------------------|---|------------------------------------------------------------------------|
| Protocol            | - | TLS Protocol                                                           |
| TLS Version         | - | TLSv1.2                                                                |
| Certificate Issuer  | - | common name - YR2<br>Organisation name - Let's Encrypt<br>Country - US |
| Validity            | - | Depend on the certificate                                              |
| Certificate Subject | - | firefox.settings.services.mozilla.com                                  |
| Cipher Suite        | - | TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256                                |
| Application Data    | - | Encrypted                                                              |





## 5.Hashing Workflow

A **hash workflow** is used to create and verify a unique fingerprint of data. In **PowerShell**, commands like `Get-FileHash` can generate a file hash, while **Python** can use the `hashlib` library to calculate hashes such as SHA-256. **CyberChef** can also generate and compare hashes easily, helping verify that a file or message has not been changed.

Use powershell, python, cyberchef tried to file hash check ...are given below

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\moham> "hello, this is my first hash" | out-file test.txt
PS C:\Users\moham> cat test.txt
hello, this is my first hash
PS C:\Users\moham> get-filehash test.txt -algorithm SHA512

Algorithm      Hash                                                    Path
-----
SHA512         A0DC9C9FB96DE48A134FA409A85BE9BFE7F349FE84BB120DBCC7257A9ADBCB0BEF... C:\Users\moham\test.txt

PS C:\Users\moham> get-filehash test.txt -algorithm SHA256

Algorithm      Hash                                                    Path
-----
SHA256         E6CC2A289025A1361338D67DFCE439A542361FE38B47421559EA2CA5286858C9 C:\Users\moham\test.txt

PS C:\Users\moham> "hello, this is my second hash" | out-file test.txt
PS C:\Users\moham> cat test.txt
hello, this is my second hash
PS C:\Users\moham> get-filehash test.txt -algorithm SHA512

Algorithm      Hash                                                    Path
-----
SHA512         EAAD059FAF29C7E4DCB877545789BF58C4193936D53392E80287DB93C652A2F606C... C:\Users\moham\test.txt

PS C:\Users\moham> get-filehash test.txt -algorithm SHA256

Algorithm      Hash                                                    Path
-----
SHA256         6A9C99031AEF4FA76D2BEFF1F80E61E0AF2AA65D19151E9EFC9B8D59B8D396D C:\Users\moham\test.txt

PS C:\Users\moham> |
```

The image shows a Visual Studio Code editor with a Python file named `hash_check.py`. The script uses the `hashlib` module to calculate SHA-256 hashes of a file named `new1.txt`. It compares the hash of the original file with the hash of the file after a modification. The terminal window shows the command `python -u "e:\python\3.hands on task\hash_check.py"` being executed, and the output shows the hash values and a message indicating the file has changed.

```
1 import hashlib
2 with open("new1.txt","rb") as file:
3     read=file.read()
4     hash_check=hashlib.sha256(read).hexdigest()
5     print("hash value : ",hash_check)
6
7 with open("new1.txt","rb") as file:
8     read=file.read()
9     hash_check2=hashlib.sha256(read).hexdigest()
10    print("hash value : ",hash_check2)
11
12 if(hash_check==hash_check2):
13     print("file does not change ")
14 else:
15     print("file is change")
```

Terminal Output:

```
PS E:\python> python -u "e:\python\3.hands on task\hash_check.py"
hash value : 025727f8029e840e128a2cde64e0963d3c56a5907ef030c440e1bd2fca55888
hash value : 025727f8029e840e128a2cde64e0963d3c56a5907ef030c440e1bd2fca55888
file does not change
PS E:\python>
```

The image shows the CyberChef web application interface. The 'Recipe' tab is active, showing a 'SHA2' recipe with a size of 512 and 160 rounds. The 'Input' tab shows the text 'this is the hash'. The 'Output' tab displays the resulting SHA-256 hash: `7baf92f840dad60d54a0dd509e98b7c0a4ba0b7d73c6cc03403b9efefbae7d60df212f2bdee8b4895cc1d34720dca9130d8bf66d4e01fbbe5a471af2301f6`.

| Algorithm | Before                                                                     | After                                                                   |
|-----------|----------------------------------------------------------------------------|-------------------------------------------------------------------------|
| SHA512    | ABDC9C9FB96DE48A134FA409A85BE9BFEE7F349FE84BB120DBC<br>C7257A9ADBCB BEF... | EAAD059FAF29C7E4DC88775457898F58C<br>4193936053392E802870893C652A2F606C |
| SHA256    | E6CC2A289025A1361338067DFCE<br>439A542361FE38847421559EA2CA<br>5286858C9   | 6A9C99031AEF4FA76028EFF1FB8E61E8A<br>F2AA65019151E9EFDC9880598503960    |

## 6. Demonstrate Encryption concepts

Encryption converts readable data (plaintext) into an unreadable form (ciphertext) using an encryption method and key.

In CyberChef, you can use operations like AES Encrypt to encrypt text and AES Decrypt to convert it back.

In Python, libraries such as `cryptography` can be used to perform encryption and decryption.

The same key or the corresponding private key is used to decrypt the encrypted data, depending on the encryption type.

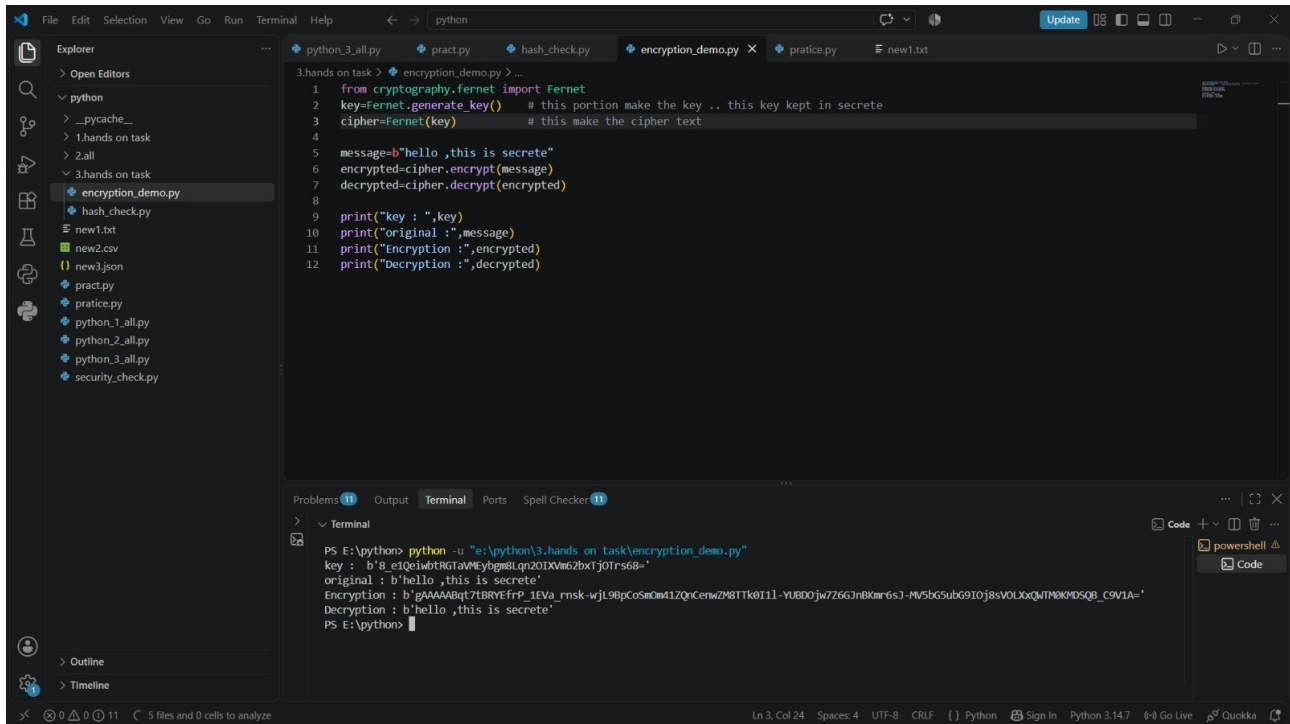
### Encryption :-

The screenshot shows the CyberChef web interface. On the left, the 'Operations' sidebar lists various encryption and decryption tools. The 'Recipe' panel in the center is configured for 'AES Encrypt'. The 'Key' is set to '71263876211...' and the 'IV' is '71263876211...', both in UTF8 format. The 'Mode' is 'CBC', 'Input' is 'Raw', and 'Output' is 'Hex'. The 'Include IV in output' checkbox is 'Off'. The 'Input' panel on the right contains the text 'this is the first hash from cyberchef'. The 'Output' panel at the bottom displays the resulting ciphertext: '937ccfaaa881e99f75da9d678041a9a8ef6bb50afc41f793086dc26dbd097cf8d32ea89b10bf16f0052d5373d662b55d'. A green 'BAKE!' button is visible at the bottom of the recipe panel.

### Decryption :-

The screenshot shows the CyberChef web interface configured for decryption. The 'Recipe' panel is set to 'AES Decrypt'. The 'Key' remains '71263876211...' in UTF8, but the 'IV' is now '1387621187687' in UTF8. The 'Mode' is 'CBC', 'Input' is 'Hex', and 'Output' is 'Raw'. The 'IV from input' checkbox is 'Off'. The 'Input' panel on the right contains the ciphertext: '937ccfaaa881e99f75da9d678041a9a8ef6bb50afc41f793086dc26dbd097cf8d32ea89b10bf16f0052d5373d662b55d'. The 'Output' panel at the bottom displays the decrypted plaintext: 'this is the first hash from cyberchef'. The 'BAKE!' button is again visible at the bottom of the recipe panel.

python use:-



```
1 from cryptography.fernet import Fernet
2 key=Fernet.generate_key() # this portion make the key .. this key kept in secreta
3 cipher=Fernet(key) # this make the cipher text
4
5 message=b"hello ,this is secreta"
6 encrypted=cipher.encrypt(message)
7 decrypted=cipher.decrypt(encrypted)
8
9 print("key :",key)
10 print("original :",message)
11 print("Encryption :",encrypted)
12 print("Decryption :",decrypted)
```

```
PS E:\python> python -u "e:\python\3.hands on task\encryption_demo.py"
key : b'8_e1Q6iW6tRGTaWMybgm8LQp20IXWm62bXtj0Trs68-'
original : b'hello ,this is secreta'
Encryption : b'gAAAAABqt7tBRVEfrP_1EVa_rnsk-wjL98pCoSmOm412QxCenwZ98ITk8I1l-YUEDOjw7Z6GJnBkmr6sJ-MV5bGSubG9IOj8sVOLxxQWtM8XMDsQ8_C9V1A-'
Decryption : b'hello ,this is secreta'
PS E:\python>
```

|                  |   |                                                                                                      |
|------------------|---|------------------------------------------------------------------------------------------------------|
| Algorithm        | - | AES                                                                                                  |
| Mode             | - | CBC                                                                                                  |
| plaintext        | - | This is the first hash from cyberchef                                                                |
| key              | - | 7126387621187687                                                                                     |
| IV               | - | 7126387621187687                                                                                     |
| Encrypted output | - | 937ccfaaa881e99f75da9d678641a9a8ef6bb50afc41f79<br>3086dc26dbd097cf8d32ea89b10bf16f0052d5373d662b55d |
| Decrypted output | - | This is the first hash from cyberchef                                                                |

## 9. Conclusion

Through these hands-on tasks, I learned how secure communication works in practice.

I observed HTTPS traffic using Wireshark, investigated TLS certificates and certificate chains, used SHA-256 hashing to detect file changes, and demonstrated encryption and decryption using Python.

These practical exercises helped me connect **cryptography and TLS theory with real-world security tools**.